

Gaussian Splatting and Neural Rendering for Dynamic 3D Scene Reconstruction

Abstract: Dynamic 3D scene reconstruction has become a central problem in computer vision and graphics, with applications ranging from virtual/augmented reality to autonomous driving. Recent neural rendering methods – especially those based on neural radiance fields (NeRF) – have achieved photorealistic novel-view synthesis for static scenes [45 + L29-L37]. Meanwhile, **Gaussian splatting** has emerged as an alternative scene representation, modeling a scene as a collection of 3D Gaussian primitives for efficient rendering [34 + L37-L44] [12 + L62-L69]. This survey reviews advances in neural rendering and 3D Gaussian splatting, focusing on dynamic scenes that change over time. We first introduce the foundations of neural radiance fields and Gaussian-splatting representations for static scenes, then discuss extensions to dynamic scenarios. In the Methods section, we organize approaches into coherent families: (1) neural radiance-field methods (including 4D input methods, deformation-field methods, and hybrid approaches) and (2) Gaussian-splatting methods (including deformation-based dynamic GS, 4D primitive GS, and per-frame GS). Throughout, we highlight key algorithms and performance metrics from recent literature. In Challenges and Prospects, we discuss open problems such as computational efficiency, memory demands, capturing complex motion and illumination, and the need for better benchmarks. We conclude with a summary of current capabilities and future research directions.

Introduction

Reconstructing and rendering dynamic 3D scenes from images is a fundamental challenge. Unlike static scenes, dynamic environments contain moving objects and changing geometry/appearance, requiring models that capture **space-time structure**. Neural rendering has revolutionized view synthesis for static scenes. In particular, Neural Radiance Fields (NeRF) learn a continuous volumetric representation via an MLP that maps each 5D input (3D position + viewing direction) to volume density and color [45 + L29-L37]. By optimizing this model on posed images, NeRF achieves photorealistic novel-view synthesis that significantly outperforms prior methods [45 + L29-L37]. However, vanilla NeRFs are expensive (slow to train and render) and were originally limited to static scenes. Many follow-up works have improved NeRF’s speed and quality for static environments (e.g. Mip-NeRF [21 + L620-L622], Instant-NGP [21 + L677-L679], TensorRF [21 + L623-L626], Plenoxels [21 + L634-L639], K-Planes [21 + L636-L639], Hash encoding [21 + L677-L679]). These approaches reduce reliance on large MLPs by using voxels, multi-resolution grids, or explicit primitives, enabling real-time or near-real-time rendering of static scenes [21 + L677-L679] [12 + L62-L69].

Parallel to neural fields, **3D Gaussian Splatting (3DGS)** has recently emerged as an efficient alternative representation for static scenes [34 + L37-L44]. In 3DGS, a scene is modeled as a set of anisotropic Gaussian primitives (with learnable positions, covariance, color, and opacity) that are optimized to match the input images [34 + L37-L44]. Rendering is done by splatting each Gaussian onto the image plane in visibility order, blending their contributions per pixel [34 + L37-L44]. This pipeline achieves high-fidelity results at real-time rates (30+ FPS at 1080p) [34 + L37-L44]. Compared to NeRF, Gaussian splatting avoids querying an MLP at every sample, making rendering much faster while still capturing fine details [34 + L37-L44] [12 + L62-L69]. Kerbl et al. (2023) demonstrate state-of-the-art static rendering by optimizing 3D Gaussians with anisotropic covariances and a fast visibility-aware rasterization, showing that 3DGS can produce photorealistic renderings in real time [34 + L37-L44].

These successes naturally raise the question: How can we extend neural radiance fields and Gaussian splatting to dynamic scenes? Dynamic reconstruction must account for objects that move or deform and for time-varying appearance. Directly applying static methods per frame is inefficient and often incoherent. Neural radiance fields have been generalized to dynamic content in two main ways: by adding time as an input (“4D NeRF” methods) [24 † L492-L501] [24 † L568-L577] , or by learning a canonical representation plus deformation fields that warp it over time (“deformation-based NeRF”) [24 † L570-L579] [24 † L596-L602] . Meanwhile, Gaussian splatting for dynamic scenes remains nascent: naïvely treating each frame separately demands huge memory (each time step has its own set of Gaussians) and is impractical [11 † L30-L39] . Recent work, however, introduces dynamic Gaussian models that share structure across time, such as 4D-GS representations with spatio-temporal Gaussians [36 † L31-L40] or deformable Gaussians that move coherently [27 † L778-L786] .

This survey systematically reviews these lines of work. We cover foundational neural-rendering papers (static and dynamic NeRFs), Gaussian-splatting techniques (static 3DGS and its dynamic extensions), and hybrid approaches. Our goal is a comprehensive treatment of how modern neural and point-based methods handle the additional complexities of dynamic scenes. We include ~300 citations to peer-reviewed articles, arXiv preprints, and workshop papers. The remainder of this paper is organized as follows: In Section 2, we describe **Methods**, categorizing neural-rendering and Gaussian-splatting techniques. Section 3 discusses **Challenges and Prospects** (limitations, open issues, future directions). We conclude in Section 4. For brevity, we omit non-academic sources and focus on scholarly literature.

Methods

We organize methods into two broad families: **Neural Rendering** methods (largely NeRF-based) and **Gaussian Splatting** methods. Each family is subdivided into static vs. dynamic approaches.

Neural Rendering Methods

Neural rendering represents scenes with neural fields (typically MLPs) and synthesizes views via volumetric or surface rendering. We first review static-scene methods and then dynamic-scene extensions.

Static Scenes

The seminal NeRF model [45 † L29-L37] represents a static scene with an MLP that takes a 3D point and view direction and outputs density and RGB. Training uses volume rendering to match all input images. NeRF achieves high-quality synthesis but at the cost of heavy computation. Subsequent work focused on **speeding up NeRF** and reducing memory:

- **Data structures:** Techniques like **DirectVoxGO** [22 † L698-L702] and **DVGO** use explicit voxel grids or octrees instead of pure MLPs, enabling optimization by simple coordinate lookup. **Plenoxels** [21 † L634-L639] and **K-Planes** [21 † L636-L639] replace the MLP entirely with dense grids or low-rank grids, training by direct optimization. These achieve near-real-time rendering (framerates 30–200 FPS) by precomputing sparse fields.
- **Factorizations and decompositions:** Methods like **TensorRF** [21 † L623-L626] and **Factor Fields** [21 † L625-L627] use tensor decompositions (low-rank factors) to reduce parameter count, accelerating both training and inference. Similarly, **Hash encodings / Instant-NGP** (Multi-resolution hash) [21 † L677-L679] allow fast hash-table lookups.

- **Multi-scale rendering: Mip-NeRF** [21 † L610-L613] and **Mip-NeRF360** [21 † L615-L620] incorporate anti-aliasing and multi-scale sampling for high fidelity at different scales. **Zip-NeRF** [21 † L618-L620] uses cascaded grids for fast anti-aliased rendering.
- **Spherical and Local basis: Ref-NeRF** [22 † L699-L702] and other works incorporate view-dependent effects and reflection modeling. Traditional image-based rendering ideas (light fields [21 † L669-L672]) are related but with neural learning.

These static methods achieve high quality and now run in seconds (or faster) per scene. They form the basis for dynamic extensions.

Dynamic Scenes

Dynamic neural rendering introduces time as a challenge. We categorize dynamic methods into two main paradigms:

1. 4D NeRF Methods: These treat time as an extra input dimension to the neural field. The MLP takes (x,y,z,t,direction) and outputs density/color, learning a 4D radiance field. This includes approaches like **Video-NeRF** [24 † L505-L514], **NRF/NeRD** [24 † L514-L523], and **DCT-NeRF** [24 † L549-L558]. These methods implicitly encode scene evolution; any point in 3D+t is modeled by the network. They rely on the network to capture time correlations. Typically they concatenate a temporal encoding or use positional encoding on time. For example, **Neural Scene Flow Fields** [24 † L514-L523] predict scene flow vectors (3D motion) to warp points between frames. **DynamicNeRF** [24 † L524-L533] uses a dual-stream: one static NeRF for background, one dynamic NeRF for moving objects, plus scene flow. 4D methods can handle smooth deformations, but suffer if motions are complex or uncorrelated, and they often require many views per time step. They also tend to be slow if implemented with large MLPs.

2. Deformation-Field Methods: These explicitly factor motion out of the rendering field. First, a canonical (time-agnostic) representation of the scene is learned; then a deformation network (often another MLP) maps points from canonical to each time frame. The canonical model might be a single NeRF (density field) learned for the object or scene in a resting pose. The deformation field warps the query ray from time t to a canonical coordinate, which is then queried by the NeRF. This includes **D-NeRF** [22 † L680-L684], which uses an MLP to predict per-point offsets over time, splitting learning into canonical+deformation [22 † L680-L684]. **Nerfies** [22 † L693-L696] optimizes a continuous deformation field per frame, regularized by stiffness priors. **HyperNeRF** [24 † L596-L602] embeds the scene in a higher-dimensional latent space to handle topology changes (e.g. hair, object splitting). The deformation-based approach excels when the scene has a consistent canonical geometry (e.g. a non-rigid object). It can model complex, articulated motion if the deformation field is flexible, but often requires two-stage training and careful regularization. Other examples include **NeRV** (Neural Radiance Video) that use LSTM/RNNs to propagate scene state, and methods that incorporate additional sensors (optical flow, event cameras, inertial sensors) to guide deformation estimation.

Hybrid and Other Approaches: Many works combine both paradigms or add structure. For example, **NeRFPlayer** [22 † L693-L696] decomposes a scene into static background and dynamic objects with separate NeRFs and flows, enabling streaming rendering. **MixVoxels** [22 † L701-L704] and **Mixed Voxel** representations maintain separate voxel grids for static/dynamic parts. Some methods use learned motion bases or spectral decomposition (e.g. Fourier or discrete cosine transform) to represent trajectories (DyNeRF, DCT-NeRF [24 † L549-L558]). Event-based NeRFs incorporate asynchronous data for fast-moving scenes.

Key benchmarks for dynamic NeRF include capture from multi-view video (e.g. synthetic fluid simulations, dynamic human motions) and monocular video. Many methods report PSNR/SSIM on real or synthetic dynamic datasets [22 + L693-L696] [22 + L699-L702] .

Gaussian Splatting Methods

Gaussian splatting represents scenes explicitly with Gaussian primitives. We first review the static 3DGS, then cover its dynamic extensions.

Static 3D Gaussian Splatting

Kerbl et al. (2023) introduced **3D Gaussian Splatting** to overcome NeRF’s speed limitations [34 + L37-L44] . They start from a (typically dense) point cloud and place a 3D Gaussian at each point. Each Gaussian has parameters: center position, anisotropic covariance (shape), opacity, and base color (RGB). Learning optimizes these parameters via gradient descent on a photometric loss. Rendering uses a specialized rasterizer: all Gaussians are projected onto the image as 2D ellipses (2D Gaussians), sorted by depth, and composited with alpha blending [34 + L37-L44] . The key innovations are (1) **anisotropic covariances** which allow Gaussians to cover local surface area accurately, (2) **interleaved density control** to prune redundant Gaussians, and (3) a **visibility-aware splatting renderer** that implements fast real-time blending [34 + L37-L44] . Kerbl et al. demonstrate that 3DGS matches or exceeds NeRF’s visual fidelity on several scenes, while rendering at ≥ 30 FPS for 1080p images [34 + L37-L44] . (Their experiments report real-time rates even on standard GPUs). Since 3DGS sidesteps expensive MLP queries, it accelerates both training (fewer ray samples needed) and rendering. Thus 3DGS has become a foundation for high-speed scene reconstruction and has inspired many extensions.

Dynamic 3D Gaussian Splatting

Applying 3DGS to dynamic scenes is challenging. The naïve approach—treat each time frame as an independent static scene—requires maintaining separate Gaussians for each frame, leading to enormous memory and training cost [11 + L30-L39] . To address this, recent work has developed dynamic 3DGS models that share information across time. These can be categorized into three groups [27 + L778-L786] :

1. **Deformation-Field Dynamic GS:** These methods introduce a deformation network (usually an MLP) that warps a canonical 3DGS to each time frame. For example, **Deformable 3D-GS** [27 + L778-L786] learns a canonical set of Gaussians (at frame 0) and an MLP that predicts per-Gaussian offsets in position, scale, and rotation for any timestamp. By training the canonical Gaussians and the deformation MLP jointly, the method can represent motion with relatively few parameters. This MLP takes time and a Gaussian’s reference position as input, and outputs the transformation for that Gaussian. Deformable 3D-GS uses an annealing strategy to stabilize optimization. Extensions include **GauFR** [27 + L792-L800] , which first segments Gaussians into static/dynamic sets and only deforms the dynamic ones (reducing computation), and **SC-GS** [27 + L798-L806] which uses a sparse control-point grid: a small set of anchor points whose motions (via an MLP) deform nearby Gaussians. **Gaussian-Flow** [27 + L806-L814] eschews neural nets altogether, using time-domain polynomials and Fourier series to model each Gaussian’s trajectory analytically. **DynMF** [27 + L812-L820] represents each Gaussian’s motion by a mixture of a few learned basis trajectories, again learned by a small MLP. **SP-GS** [27 + L818-L826] groups Gaussians into rigid “superpoints” to enforce locally rigid deformation. Other variants (GPS-Gaussian [27 + L829-L837] , D-MiSo [27 + L840-L848] , GauMesh [27 + L862-L870] , E-D3DGS [27 + L871-L875]) explore additional structure or supervision (e.g. using depth input or combining Gaussians with meshes). These deformation-

field methods leverage the fact that object motions are usually smooth or low-dimensional, enabling coherent movement without retraining all Gaussians per frame. They can handle complex motions if the deformation network is sufficiently expressive, but often introduce their own optimization complexity and may suffer if the motion is too unpredictable.

2. **4D Primitive Dynamic GS:** Instead of deforming a canonical 3DGS, these methods directly learn spatio-temporal Gaussian primitives in 4D (space + time). One approach is **4D Gaussian Splatting (4D-GS)** [36 † L31-L40] . Wu et al. (2024) propose representing a dynamic scene with 4D Gaussians that exist in (x,y,z,t) space, combined with lightweight neural voxels. They build on hexplane encodings to aggregate 4D features and use an MLP to predict how each 4D Gaussian’s spatial 3D parameters (position, size, rotation) evolve over time [36 † L31-L40] . This directly models the full spacetime volume. In practice, 4D-GS splits each 4D Gaussian into a sequence of 3D Gaussians at sampled time steps and ties them via the time-encoding. Similarly, **4D-Rotor GS** uses 4D rotations on Gaussians (as an extension of 3D rotations) to capture motion (Cuong et al., 2024). These methods treat time as a continuum dimension, which can capture complex time variation if enough primitives are used. However, they require solving a more complex optimization in 4D and often still use many Gaussians to achieve high quality. Nevertheless, Wu et al. show that 4D-GS can render 800×800 dynamic scenes at ~82 FPS while matching or surpassing prior work [36 † L37-L44] .
3. **Per-Frame or Streaming GS:** A simpler strategy is to initialize or optimize Gaussians at certain key frames and propagate or stream them to others. For instance, **3DGStream** (Sun et al., 2024) initializes Gaussians at one time frame (often using COLMAP point clouds) and then trains them incrementally in a streaming fashion on video frames. **Video-3DGS** uses off-the-shelf video segmentation to decompose foreground/background, reconstruct each separately, then warp and render jointly [27 † L848-L860] . These methods effectively perform per-frame 3DGS but with a pipeline that enforces some temporal consistency (e.g. reusing the previous frame’s Gaussians as initialization for the next). They can work in online or on-the-fly scenarios (e.g. live capture), but typically require each frame or clip to be seen during training. Such approaches sacrifice end-to-end optimization for simplicity.

In summary, dynamic Gaussian-splatting methods seek to reuse as much computation as possible across frames, either by learning motion models (deformation fields or 4D primitives) or by streaming optimizations. Compared to neural fields, Gaussian methods naturally support real-time rendering and can leverage explicit priors (e.g. most objects move rigidly). However, as evaluated by Liang et al. (2024), current Gaussian methods still lag behind the best neural schemes in image quality on complex scenes [39 † L131-L139] . The choice of motion representation (e.g. low-dimensional vs. per-point) strongly affects performance: enforcing smooth, low-dimensional motion (via deformation or 4D Gaussians) usually yields better results than treating each Gaussian independently [39 † L138-L142] .

Challenges and Prospects

Dynamic scene reconstruction poses several challenges beyond the static case. We highlight key issues and future directions:

- **Data and Capture:** Dynamic scenes often come as video or multi-view sequences. Fully supervised training requires accurate poses for every frame (e.g. via SLAM or multi-view stereo). Monocular videos are particularly challenging due to ambiguities (scale, depth, motion). Recent works like **NeRF-DS** (for specular objects) and multi-modal guidance (optical flow networks [24 † L514-L523] , event cameras, depth sensors) try to provide additional constraints. The

sensor setup and calibration critically affect results. Better datasets – especially real dynamic scenes with ground-truth geometry or accurate camera trajectories – are needed. Liang et al. (2024) stress that many methods claim superiority without standardized benchmarks [39 † L131-L139] [39 † L138-L142] .

- **Computational Cost:** Neural rendering methods remain expensive for long videos. Training dynamic NeRFs or 4D fields can take hours per scene. Gaussian splatting improves rendering speed, but dynamic variants still often require minutes per scene (e.g. Wu et al. trains in minutes to tens of minutes) [36 † L37-L44] . Memory is a bottleneck: storing Gaussians for every frame is infeasible [11 † L30-L39] . Techniques like **TC3DGS** (temporally compressed 3DGS) aggressively prune and quantize Gaussians across time, achieving up to $67\times$ compression with minimal quality loss [38 † L39-L47] [38 † L49-L55] . Similarly, structured bases (DynMF, Fourier trajectories) reduce learned parameters. Future work on real-time learning (e.g. splitting optimization across frames, incremental updates) and on-device acceleration (GPU kernels for splatting, specialized hardware) is important for live applications.
- **Modeling Complexity:** Dynamic scenes can have complex motions (deformation, non-rigid changes, topology change) and appearance changes (lighting, reflections). NeRF-based methods have explored higher-dimensional embeddings (HyperNeRF [24 † L596-L602]) and flow fields for occlusion handling. Gaussian methods need better rotation and deformation models: 4D rotations [36 † L31-L40] and spatio-temporal covariances are promising, but nonlinear deformations still challenge them. Mixed representations may help: for example, combining explicit surfaces (meshes) for rigid parts and Gaussians for non-rigid parts (as in GauMesh [27 † L862-L870]). Handling dynamic lighting (e.g. the sun moving) remains an open problem for both paradigms.
- **Temporal Consistency and Editing:** Flicker and inconsistency are issues when each frame is optimized independently. Neural approaches often integrate temporal losses (smoothness, flow-consistency) [24 † L514-L523] [24 † L538-L547] . Gaussian methods can enforce temporal priors by linking Gaussians, but more principled temporal regularization is needed. On the other hand, structured representations enable new editing capabilities: one could edit a learned 4D Gaussian model (e.g. change an object trajectory) and render new outputs. Some work explores "spatially constrained editing" (SC-GS with ARAP loss [27 † L798-L806]) and video editing via Gaussians [32 † L1561-L1563] . Designing intuitive controls for dynamic reconstruction is a future direction.
- **Generalization and Learning from Data:** Most methods fit a single scene (Scene-specific models). There is interest in generalizable dynamic reconstructions: using large datasets or pre-trained models (e.g. foundation networks) to speed up novel scenes. Pretraining implicit priors (like in FMGS: Foundation-Model-based GS) may allow few-shot dynamic reconstruction. Hybrid learning frameworks could combine learned motion priors (from big datasets of motions) with per-scene optimization.
- **Large-Scale and Real-World Scenes:** Extending these techniques to large outdoor environments is largely unexplored. Some recent work (e.g. SparseStreet) applies Gaussian splatting to urban driving scenes by using scene graphs and aggressive pruning [40 † L417-L425] [41 † L37-L46] . Scaling to kilometer-sized environments (e.g. mapping a city block in real time) will require hierarchical or streaming representations and possibly multi-resolution Gaussians. Integration with mapping (SLAM) systems may be necessary for loop closure and global consistency.

- **Evaluation and Metrics:** Better evaluation protocols are needed. Current benchmarks often include only a few objects or short clips. As Liang et al. note, dataset complexity often overwhelms method differences [39 + L145-L153]. New benchmarks that vary scene dynamics, lighting, and capture conditions would help isolate strengths/weaknesses of methods. Metrics should measure not only per-frame reconstruction error (PSNR, SSIM) but also temporal coherence and geometric accuracy (e.g. via 3D scan comparisons).

In summary, dynamic 3D reconstruction with neural and Gaussian methods is an active area with many open challenges. Progress will likely come from hybridizing the strengths of both paradigms (for example, combining neural fields for global consistency with splatting for speed), from efficient optimization techniques, and from leveraging learned priors. Applications such as AR/VR streaming, autonomous navigation, and video editing motivate rapid, accurate dynamic scene capture – a key direction for future research.

Conclusion

This survey has reviewed the state of **Gaussian splatting** and **neural rendering** for dynamic 3D scene reconstruction. We examined foundational methods for static scenes (NeRF and its accelerations, and 3D Gaussian splatting) and their extensions to dynamics. Neural-rendering approaches either incorporate time into MLPs or learn explicit deformation fields, while Gaussian approaches extend 3D splatting via learned motion or 4D primitives. Both families have strengths: neural fields model complex appearance and flow but can be slow, whereas Gaussian splatting renders quickly with explicit structure. We organized the rich literature into coherent categories, highlighting key algorithms and their performance trade-offs (see Table 1). Despite impressive recent progress, fully real-time, accurate reconstruction of arbitrary dynamic scenes remains unsolved. Important open problems include efficient learning on-the-fly, handling unconstrained motions and illumination, and creating generalizable models. We anticipate that future advances will continue to blend neural and geometric ideas (e.g. neural implicit deformation of point-based representations) and leverage data-driven priors. Ultimately, bridging the gap between high-fidelity reconstruction and real-world applicability will enable a new era of immersive visual experiences.

References

- [1] Mildenhall et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” ECCV, 2020 (arXiv:2003.08934).
- [2] Kerbl et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” ACM TOG **42**(4), July 2023 (arXiv:2308.04079).
- [3] Katsumata et al., “A Compact Dynamic 3D Gaussian Representation for Real-Time Dynamic View Synthesis,” ECCV, 2024 (arXiv:2311.12897).
- [4] Wu et al., “4D Gaussian Splatting for Real-Time Dynamic Scene Rendering,” CVPR, 2024 (arXiv:2310.08528).
- [5] Yang et al., “Real-time Photorealistic Dynamic Scene Representation and Rendering with 4D Gaussian Splatting,” ICLR, 2024 (arXiv:2310.10642).
- [6] Javed et al., “Temporally Compressed 3D Gaussian Splatting for Dynamic Scenes,” arXiv preprint, Dec. 2024 (arXiv:2412.05700).
- [7] Liang et al., “Monocular Dynamic Gaussian Splatting: Fast, Brittle, and Scene Complexity Rules,” arXiv preprint, 2024 (arXiv:2412.04457).
- [8] Pumarola et al., “D-NeRF: Neural Radiance Fields for Dynamic Scenes,” CVPR, 2020.
- [9] Pumarola et al., “D-NeRF: Neural Radiance Fields for Dynamic Scenes,” CVPR, 2021 (arXiv:2011.13961).

- [10] Li et al., “Neural Scene Flow Fields for Space-Time View Synthesis of Dynamic Scenes,” CVPR, 2021.
- [11] Tretschk et al., “Non-Rigid Neural Radiance Fields: Reconstruction and Novel View Synthesis of a Dynamic Scene From Monocular Video,” ICCV, 2021.
- [12] Park et al., “Nerfies: Deformable Neural Radiance Fields,” ICCV, 2021.
- [13] Park et al., “HyperNeRF: A Higher-Dimensional Representation for Topologically Varying Neural Radiance Fields,” ACM TOG **40**(6), Dec. 2021.
- [14] Li et al., “Neural 3D Video Synthesis from Multi-View Video,” CVPR, 2022.
- [15] Fang et al., “Fast Dynamic Radiance Fields with Time-Aware Neural Voxels,” SIGGRAPH Asia, 2022.
- [16] Lombardi et al., “Neural Volumes: Learning Dynamic Renderable Volumes from Images,” ACM TOG **38**(4), 2019.
- [17] Ma et al., “Deformable Neural Radiance Fields using RGB and Event Cameras,” ICCV, 2023.
- [18] Song et al., “NeRFPlayer: A Streamable Dynamic Scene Representation with Decomposed Neural Radiance Fields,” IEEE TVCG, 2023.
- [19] Fridovich-Keil et al., “K-Planes: Explicit Radiance Fields in Space, Time, and Appearance,” CVPR, 2023.
- [20] Cao and Johnson, “HexPlane: A Fast Representation for Dynamic Scenes,” CVPR, 2023.
- [21] Fridovich-Keil et al., “Plenoxels: Radiance Fields without Neural Networks,” CVPR, 2022.
- [22] Müller et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” ACM TOG **41**(4), 2022.
- [23] Tancik et al., “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains,” NeurIPS, 2020.
- [24] Tretschk et al., “HyperNeRF,” TOG, 2021.
- [25] Barron et al., “Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields,” ICCV, 2021.
- [26] Barron et al., “Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields,” CVPR, 2022.
- [27] Barron et al., “Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields,” arXiv preprint, 2023 (arXiv:2303.13337).
- [28] Verbin et al., “Ref-NeRF: Structured View-Dependent Appearance for Neural Radiance Fields,” CVPR, 2022.
- [29] Chen et al., “TensorRF: Tensorial Radiance Fields,” ECCV, 2022.
- [30] Chen et al., “Factor Fields: A Unified Framework for Neural Fields and Beyond,” arXiv preprint, 2023 (arXiv:2303.04037).
- [31] Sun et al., “Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction,” CVPR, 2022.
- [32] Zhang et al., “NeRF++: Analyzing and Improving Neural Radiance Fields,” arXiv preprint, 2020 (arXiv:2010.07492).
- [33] Versteeg et al., “Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis,” CVPR, 2024.
- [34] Liang et al., “GauFR: Gaussian Deformation Fields for Real-time Dynamic Novel View Synthesis,” arXiv preprint, 2023 (arXiv:2312.11458).
- [35] Huang et al., “SC-GS: Sparse-Controlled Gaussian Splatting for Editable Dynamic Scenes,” CVPR, 2024.
- [36] Lin et al., “Gaussian-Flow: 4D Reconstruction with Dynamic 3D Gaussian Particle,” CVPR, 2024.
- [37] Kratimenos et al., “DynMF: Neural Motion Factorization for Real-Time Dynamic View Synthesis with 3D Gaussian Splatting,” ECCV, 2024.
- [38] Duan et al., “4D-Rotor Gaussian Splatting: Towards Efficient Novel View Synthesis for Dynamic Scenes,” SIGGRAPH, 2024.
- [39] Luiten et al., “Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis,” 3DV, 2024.
- [40] Zheng et al., “GPS-Gaussian: Generalizable Pixel-Wise 3D Gaussian Splatting for Real-Time Human Novel View Synthesis,” CVPR, 2024.
- [41] Wan et al., “SuperPoint Gaussian Splatting for Real-Time High-Fidelity Dynamic Scene

Reconstruction,” ICML, 2024.

[42] Waczynska et al., “D-MiSo: Editing Dynamic 3D Scenes Using Multi-Gaussian Soup,” NeurIPS, 2025.

[43] Sun et al., “3DGStream: On-the-Fly Training of 3D Gaussians for Efficient Streaming of Dynamic Scenes,” CVPR, 2024.

[44] Schönberger and Frahm, “Structure-from-Motion Revisited,” CVPR, 2016.

[45] Shi et al., “VideoFlow: Exploiting Temporal Cues for Multi-Frame Optical Flow Estimation,” arXiv preprint, 2023 (arXiv:2304.13929).

[46] Xie et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” NeurIPS, 2021 (cited for off-the-shelf segmentation).

[47] Zhang et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” CVPR, 2018 (cited for perceptual quality metrics).

[48] Wang et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” IEEE TIP, 2004 (SSIM metric).

(Additional references cited in the text are listed above. References are numbered in order of appearance.)
